



Pipelined RISC-V Simulator Supporting Hazard Detection and Data Forwarding

Members

Christian Watts

Alondra Conchas-Sanchez



Lab SUMMARY

In this lab, we extended our pipelined RISC-V simulator from Lab 3 to include a mechanism that is able to detect and resolve data hazards. We implemented two modes:

1. Stall-only mode, which introduces pipeline bubbles upon detecting a RAW hazard
2. Forwarding mode, which uses data forwarding paths from both the EX/MEM and MEM/WB registers to the EX stage, reducing stalls.

1. Background

A classic five stage pipeline can suffer data hazards when an instruction in ID needs a result not yet written back by an earlier instruction. Lab 4 defined four stall conditions based on registers in EX/MEM and MEM/WB matching the source registers of the ID/EX stage. Forwarding paths allow the EX stage to receive the correct operand as soon as it's available, either at the end of EX/MEM or MEM/WB, avoiding extra cycles.

2. Design and Implementation

1. Global Control of Forwarding

- a. A global flag `ENABLE_FORWARDING` defaults to 0 (stall only). Via the `f [ 0 | 1 ]` command in `handle_command()`, we set `ENABLE_FORWARDING` to 1 to activate forwarding

2. Data Hazard Detection & Pipeline Stalls

- a. In the ID stage, after decoding, we compare the destination registers in EX/MEM and MEM/WB against the source registers of the instruction in IF/ID. If a hazard is detected, we:
 - i. Set a bubble flag
 - ii. Flush ID_EX to a NOP (zero all fields)
 - iii. Freeze IF by preventing PC and IF_ID updates
- b. This ensures the dependent instruction remains in ID until its operand is safely written back

3. Data Forwarding

- a. When `ENABLE_FORWARDING == 1`, the EX stage first checks EX/MEM for a matching destination register and, if found, forwards that stage's ALU output into the operand. Failing that, it checks MEM/WB and, if appropriate, forwards either its ALU output or loaded memory data.

3. Testing and Results

We exercised both modes on the provided test files containing lw, add, and sw sequences.

1. Stall-only mode inserted one bubble per load-use hazard and correctly stalled IF, verified by observing the pipeline printout showing NOPs flowing through.
2. Forwarding mode eliminated load-use stalls except those unavoidable (e.g. uses in the immediate next cycle), reducing total cycles
3. Program output correctness was confirmed by comparing final register dumps and memory dumps

4. Work Distribution

We collaborated jointly and divided each aspect of this lab evenly, design of hazard-detection logic, implementation of pipeline stalls, data-forwarding paths, testing with sample programs, and report preparation.